

AI家AI户 API统一中转站 — 产品需求文档（PRD）

版本：v1.0 | 日期：2026-07-02 | 作者：myaishome.com 产品技术团队

本文档定义 API中转站功能的前后端改造范围、详细开发步骤、测试计划和部署方案。

目录

- [1. 产品概述与目标](#)
- [2. 用户故事](#)
- [3. 功能需求全景图](#)
- [4. 前端改造清单](#)（逐文件详细说明）
- [5. 后端新建清单](#)（逐模块详细说明）
- [6. 数据库设计](#)
- [7. API接口设计](#)
- [8. 开发步骤与里程碑](#)（6周分解，精确到天）
- [9. 测试计划](#)
- [10. 部署与运维方案](#)
- [11. 风险与应对措施](#)
- [12. 附录](#)

1. 产品概述与目标

1.1 现状

AI家AI户（myaishome.com）当前是一个**纯静态AI模型信息聚合站**：

维度	现状
页面数	16个HTML页面（首页 + 模型库 + 对比 + 技能包 + 选型器等）
模型数	44个模型详情页，6大品类
技术栈	纯HTML + CSS + JS + SVG图标，Cloudflare Pages部署
后端	零后端 ，无用户系统，无数据库
变现	仅Google AdSense，月收入可忽略不计

1.2 转型目标

在现有静态站基础上，新增 **API统一中转站（API Hub）** 模块，实现从"信息聚合"到"服务提供"的升级。

核心价值主张：一个API Key，调用所有主流AI模型。

1.3 产品定位

信息聚合（现有）

API中转站（新增）

用户价值	"告诉我哪个AI好用"	"帮我用好所有AI"
用户行为	浏览 → 离开	浏览 → 试用 → 充值 → 持续使用
收入模式	广告（极低）	API差价（可持续）
技术复杂度	静态站	全栈（前端SPA + Worker + 数据库 + 模型路由）

1.4 成功指标（MVP阶段）

- 上线4周内注册用户 ≥ 100人
- 付费转化率 ≥ 15%（即15人充值）
- 月均API调用量 ≥ 10万次
- 系统可用性 ≥ 99.5%

2. 用户故事

2.1 核心用户画像

小王（独立开发者）

"我在做一个AI写作助手，想同时支持GPT-4o和Claude，但每个平台都要单独注册充值太麻烦。如果一个平台能一次充值、一个Key调用所有模型就好了。"

小李（产品经理）

"我想试试不同模型的效果，但不知道哪个适合我的场景。有个地方能在线测试就好了。"

大刘（大学生）

"研究生论文需要调AI辅助，但海外平台支付麻烦，也没有信用卡。支持支付宝的话我马上就充值。"

2.2 关键用户旅程

旅程一：开发者快速接入

1. 在 myaishome.com 浏览模型对比 → 看中 gpt-4o
2. 点击模型旁的「获取API」→ 进入 /playground?model=gpt-4o
3. 在 Playground 输入测试 prompt → 看到流式输出结果
4. 满意 → 点击「注册获取Key」→ 输入邮箱 + 密码（20秒）
5. 跳出充值引导 → 支付宝扫码 ¥9.9
6. 系统自动生成 API Key → 一键复制
7. 打开 VSCode → 改 base_url → 开始调 API

旅程二：老用户日常使用

1. 登录 Dashboard → 看到余额 ¥23.50，本月用了 1.2M tokens
2. gpt-4o-mini 用完了很多额度 → 切换到 deepseek-chat 省钱
3. 代码里只改 model 参数: "gpt-4o" → "deepseek-chat"
4. 余额快用完 → 点「充值」→ ¥99 到账
5. 创建第二把 API Key → 给另一个项目用

旅程三：模型探索者

- 1. 看到首页「开发者API」入口 → 进入 /api
- 2. 浏览价目表 → 发现 deepseek-chat 很便宜
- 3. 点「在线测试」→ 跳转 Playground
- 4. 输入一段中文 → 效果不错
- 5. 注册充值 → 开始使用

3. 功能需求全景图



4. 前端改造清单

4.1 现有页面改造

4.1.1 index.html — 首页

改造内容:

位置	改动	说明
Hero区域	新增API CTA按钮	"开发者API" + "在线测试"，双按钮
导航栏	新增链接	「API」（指向 /api.html）

Hero区域新增代码结构：

```
<!-- 在现有Hero下方新增API CTA区块 -->
<section class="hero-api-cta">
  <h2>一个Key，调用所有AI</h2>
  <p>无需在每个平台注册。充值一次，调用 GPT-4o、Claude、DeepSeek、Gemini 等主流模型。</p>
  <div class="cta-buttons">
    <a href="/api.html" class="btn btn-primary">开发者API</a>
    <a href="/playground.html" class="btn btn-outline">在线测试</a>
  </div>
</section>
```

工作量：约30行HTML + 20行CSS，0.5天。

4.1.2 models.html — 模型库

改造内容：

位置	改动	说明
每张模型卡片	新增「试用API」按钮	跳转 /playground.html?model=xxx
顶部筛选栏	新增「支持API中转」筛选项	仅显示已接入中转站的模型

「试用API」按钮设计：

```
<a href="/playground.html?model=gpt-4o" class="btn btn-sm btn-api-try">
  <svg><use href="img/icons.svg#icon-api"/></svg> 试用API
</a>
```

筛选栏新增：

```
<label class="filter-checkbox">
  <input type="checkbox" id="filter-api" onchange="filterModels()">
  仅显示API可用
</label>
```

需要修改的数据结构（js/models-data.js 或内嵌数据）： 每个模型对象新增字段 has_api: true/false，MVP阶段5个模型为 true。

工作量：修改HTML 20行 + JS筛选逻辑 15行 + 数据字段新增 44条，1天。

4.1.3 models/*.html ×44 — 模型详情页

改造内容：

在详情页固定位置新增「试用API」CTA区块（仅 has_api: true 的模型显示）：

```
<!-- 模型详情页API CTA区块 -->
<div class="api-cta-card" id="api-cta" style="display:none">
  <h3>通过API调用此模型</h3>
  <p>无需注册此平台。使用 myaishome API Key 即可调用。</p>
  <a href="/playground.html?model=GPT-4o" class="btn btn-primary">在线测试</a>
  <a href="/api.html" class="btn btn-outline">查看API文档</a>
</div>
```

是否显示：通过JS判断URL参数中的模型名是否在API可用列表中。

工作量：44个页面各加一段HTML，需通过Python脚本批量生成。已有 scripts/generate_model_pages.py，修改模板即可。1天（含脚本修改+重新生成）。

4.1.4 compare.html — 横向对比

改造内容：

在对比表新增「API中转」行：

```
<tr>
  <th>API中转</th>
  <td><span class="badge badge-green">已接入</span></td>
  <td><span class="badge badge-green">已接入</span></td>
  <td><span class="badge badge-red">未接入</span></td>
</tr>
```

工作量：约15行HTML + 补充数据，0.5天。

4.1.5 privacy.html — 隐私政策

新增条款：

- API Key 的存储方式（仅存哈希，不存明文）
- 用户请求内容不记录（仅记录 token 数量用于计费）
- 上游API内容审核说明

工作量：约200字新增内容，0.5天。

4.2 新增页面

4.2.1 api.html — API Hub（静态页）

定位：面向开发者的API文档 + 价目表 + 代码示例。

页面结构：

 API统一中转站
一个Key，调用全球主流AI模型
[开始使用] [查看文档]

 价目表（按模型）

模型	输入	输出	单位	操作
GPT-4o	¥5	¥20	/1M tk	测试
DeepSeek	¥2	¥4	/1M tk	测试

 快速开始（3个Tab: curl / Python / Node.js）
[代码示例区域，带复制按钮]

 充值方案

¥9.9	¥39	¥99
------	-----	-----

尝鲜

+¥6

+¥20

?

FAQ

支持哪些模型？怎么计费？Key安全吗？

技术方案：纯静态HTML + JS（代码示例Tab切换 + 复制到剪贴板），无后端依赖。

工作量：全新页面设计 + 实现，约2天。

4.2.2 auth.html — 登录/注册（静态HTML + JS）

定位：轻量级认证页面，通过JS调用Worker API完成注册/登录。

页面结构：

登录 / 注册（Tab切换）

登录

邮箱：[]

密码：[]

[登录]

没有账号？去注册

注册

邮箱：[]

密码：[]

确认密码：[]

[注册]

已有账号？去登录

技术细节：- 表单提交 → POST /api/auth/register 或 POST /api/auth/login - 登录成功后 → Worker返回JWT token → 存入localStorage - 注册成功后 → 自动登录 → 跳转到充值页或Dashboard - 密码在客户端SHA-256后再传输（加salt）

工作量：约250行HTML+JS，1天。

4.2.3 playground.html — 在线测试SPA

定位：用户无需写代码即可在线测试模型，为核心转化入口。

技术方案：因需要状态管理（聊天历史、模型切换、登录状态），使用React+Vite构建SPA，打包后嵌入Pages。

页面布局：

Playground

[登录/用户]

配置面板

聊天区域

模型：
[GPT-4o ▼]

User: 帮我写一个排序算法

温度:
[0.7 

AI: 好的, 以下是...
(流式输出, 逐字显示)

Max Tokens:
[2048]

[输入框]

[发送] [清空]

系统提示:
[]

剩余试用:
2/3 次
[注册解锁]

未登录用户: 剩余3次免费试用
登录用户: 显示余额

功能清单:

功能	说明
模型选择	下拉菜单, 显示模型名+价格
系统提示	可选手动输入 System Prompt
参数调节	Temperature (滑块)、Max Tokens (输入框)
聊天区域	支持多轮对话, 流式输出 (SSE)
未登录限制	localStorage记录, 最多3次免费调用
注册引导	3次后用完后弹出注册/充值引导
对话导出	复制对话内容或下载JSON (V1功能)

React组件树:

```
PlaygroundApp
├── Header (导航 + 登录状态)
├── Sidebar
│   ├── ModelSelector (模型下拉 + 价格展示)
│   ├── ParamControls (温度滑块 + Max Tokens输入)
│   └── SystemPromptInput
├── ChatArea
│   ├── MessageList (消息列表, 支持markdown渲染)
│   │   └── MessageBubble × N (用户/AI气泡)
│   └── ChatInput (输入框 + 发送按钮)
└── TrialBanner (剩余次数 / 注册引导)
```

关键技术点: - SSE 流式读取: `fetch('/api/v1/chat/completions', {stream: true})` - Markdown渲染: 使用 `marked.js` 轻量库 - 代码高亮: 使用 `highlight.js` - 状态管理: `React Context + useReducer` (避免引入`Redux`)

工作量: 全新React项目搭建 + 6个组件开发 + 样式, 约3天。

4.2.4 dashboard.html — 用户仪表盘SPA

定位: 登录后的个人中心, 管理API Key、查看用量、充值。

技术方案: 同React SPA, 打包后嵌入Pages。

页面布局:

Dashboard

user@email.com [退出]

余额

本月用量

Key数

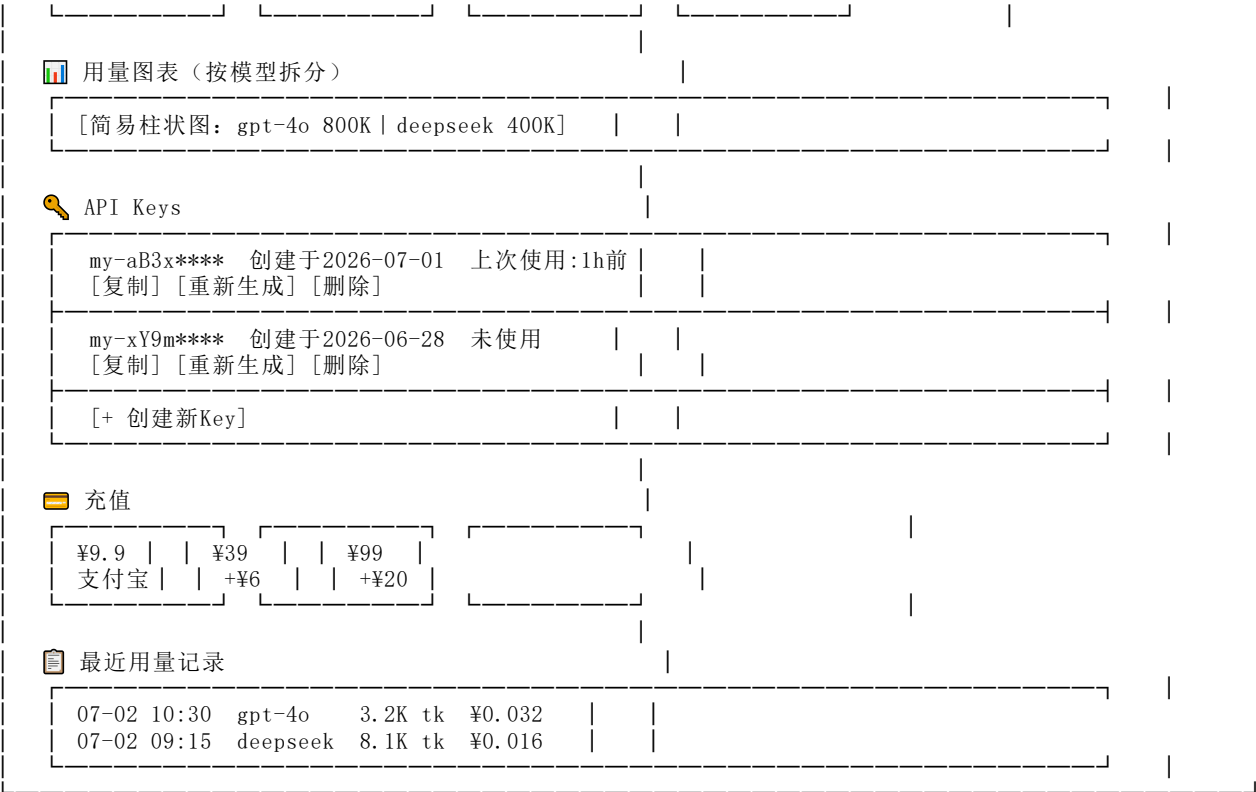
充值

¥23.50

1.2M

2

[充值]



功能清单：

功能	说明
余额展示	实时显示当前余额（Worker API查询）
用量统计	本月token总量 + 按模型拆分（简易柱状图）
API Key管理	查看（仅前缀）、复制、重新生成、删除、创建
充值入口	三档充值卡片，点击跳转支付
用量记录	最近20条调用记录（模型、时间、token量、费用）
快速充值	余额不足时页面顶部红色横幅提醒

React组件树：

```
DashboardApp
├── Header（用户邮箱 + 退出按钮）
├── StatsCards（4张统计卡片）
├── UsageChart（简易柱状图，Chart.js或纯CSS）
├── ApiKeyManager
│   ├── ApiKeyCard × N
│   └── CreateKeyButton
├── TopupSection（充值档位卡片）
└── UsageLogTable（最近用量表格）
```

工作量：全新React项目 + 7个组件 + 与Worker API联调，约3天。

4.3 前端改造汇总

页面	类型	工作量	依赖
index.html	改造	0.5天	无
models.html	改造	1天	数据字段新增

页面	类型	工作量	依赖
models/*.html	×44 批量改造	1天	Python脚本
compare.html	改造	0.5天	数据字段新增
privacy.html	改造	0.5天	条款内容确认
api.html	新建静态页	2天	SVG图标
auth.html	新建静态页	1天	Worker Auth API
playground.html	新建React SPA	3天	Worker Chat API
dashboard.html	新建React SPA	3天	Worker全部API
合计		12.5天	

5. 后端新建清单

5.1 Cloudflare Worker（API网关）

技术栈：TypeScript + Hono框架（轻量级Web框架，专为Worker设计）

项目结构：

```
ai-home-worker/
├── wrangler.toml           # CF Worker配置
├── package.json
├── tsconfig.json
├── src/
│   ├── index.ts           # 主入口：路由注册 + CORS
│   ├── auth.ts            # 鉴权中间件
│   ├── routes/
│   │   ├── v1/
│   │   │   ├── chat.ts    # POST /api/v1/chat/completions
│   │   │   ├── models.ts  # GET /api/v1/models
│   │   ├── auth.ts        # POST /api/auth/register + login
│   │   ├── keys.ts        # GET/POST/DELETE /api/user/keys
│   │   ├── billing.ts     # GET /api/user/balance + usage
│   │   └── topup.ts       # POST /api/topup/*（支付回调）
│   ├── db/
│   │   ├── schema.sql    # D1建表语句
│   │   ├── users.ts      # 用户CRUD
│   │   ├── keys.ts       # API Key CRUD
│   │   ├── billing.ts    # 余额/用量查询
│   │   └── topups.ts     # 充值记录
│   ├── services/
│   │   ├── billing.ts    # 计费引擎：token → 费用计算
│   │   ├── ratelimit.ts  # 频率限制（D1 + KV实现）
│   │   └── proxy.ts      # LiteLLM转发 + 流式透传
│   └── utils/
│       ├── crypto.ts     # 密码哈希 + API Key生成
│       ├── jwt.ts        # JWT签发/验证
│       └── errors.ts     # 统一错误响应
```

5.1.1 路由总览

路由	方法	鉴权	说明
/api/v1/models	GET	API Key	返回可用模型列表
/api/v1/chat/completions	POST	API Key	聊天补全（流式）
/api/auth/register	POST	无	用户注册
/api/auth/login	POST	无	用户登录（返回JWT）

路由	方法	鉴权	说明
/api/user/keys	GET	JWT	查看所有Key
/api/user/keys	POST	JWT	创建新Key
/api/user/keys/:id	DELETE	JWT	删除Key
/api/user/balance	GET	JWT	查询余额
/api/user/usage	GET	JWT	查询用量
/api/topup/callback	POST	PayJS签名	支付回调
/api/health	GET	无	健康检查

5.1.2 核心模块详细设计

A. 鉴权中间件 (auth.ts)

两种鉴权模式：

1. API Key 鉴权 (用于 /api/v1/*):

```
// 从 Authorization header 提取 Key
// SHA-256(Key) → 查 D1 api_keys 表 → 验证 is_active → 注入 user 到 context
```

1. JWT 鉴权 (用于 Dashboard API):

```
// 从 Cookie 或 Authorization 提取 JWT
// 验证签名 + 过期时间 → 注入 user 到 context
```

1. Guest 鉴权 (Playground试用):

```
// 前端传 X-Guest-Id (localStorage 生成的 UUID)
// Worker 查 D1 临时表或 KV, 限制 3 次
```

B. 计费引擎 (billing.ts)

```
// 模型定价表 (硬编码, 后续放 D1 或 KV)
const PRICING = {
  'gpt-4o': { input: 0.005, output: 0.020 },           // 元/1K tokens
  'gpt-4o-mini': { input: 0.0003, output: 0.0012 },
  'deepseek-chat': { input: 0.002, output: 0.004 },
  'claude-3.5-sonnet': { input: 0.006, output: 0.030 },
  'gemini-2.5-pro': { input: 0.005, output: 0.020 },
};

function calculateCost(model: string, promptTokens: number, completionTokens: number): number {
  const p = PRICING[model];
  return (promptTokens * p.input + completionTokens * p.output) / 1000;
}
```

C. 模型代理 (proxy.ts)

```
// 转发请求到 LiteLLM
async function proxyToLiteLLM(request: Request): Promise<Response> {
  const litellmUrl = 'https://litellm.internal.myaishome.com'; // 内网地址
  const response = await fetch(litellmUrl + '/v1/chat/completions', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: request.body, // 透传
  });

  // 如果是流式, 需要同时读取 SSE 并计费
  if (isStreaming) {
    return handleStreamingResponse(response, userId, model);
  }
}
```

```
// 非流式：解析 usage → 扣费 → 记录日志 → 返回
const data = await response.json();
await deductAndLog(userId, model, data.usage);
return Response.json(data);
}
```

D. 流式响应处理

这是技术难点，需要边转发 SSE 边累计 token 数量：

```
async function handleStreamingResponse(
  upstreamResponse: Response,
  userId: string,
  model: string
): Promise<Response> {
  const reader = upstreamResponse.body.getReader();
  let totalTokens = 0;

  const stream = new ReadableStream({
    async start(controller) {
      while (true) {
        const { done, value } = await reader.read();
        if (done) {
          // 流结束后统一扣费
          await deductAndLog(userId, model, { completion_tokens: totalTokens });
          controller.close();
          return;
        }
        // 解析 SSE chunk，累计 token
        const text = new TextDecoder().decode(value);
        totalTokens += extractTokensFromSSE(text);
        controller.enqueue(value);
      }
    }
  });

  return new Response(stream, {
    headers: { 'Content-Type': 'text/event-stream' }
  });
}
```

5.2 LiteLLM（模型路由）

部署目标：一台VPS（阿里云香港/腾讯云新加坡，2C4G，月费约¥50-80）。

docker-compose.yml:

```
version: '3.8'
services:
  litellm:
    image: ghcr.io/berriai/litellm:main-latest
    ports:
      - "4000:4000"
    volumes:
      - ./litellm_config.yaml:/app/config.yaml
    environment:
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - DEEPSEEK_API_KEY=${DEEPSEEK_API_KEY}
      - GEMINI_API_KEY=${GEMINI_API_KEY}
    restart: always
```

litellm_config.yaml:

```
general_settings:
  master_key: ${LITELLM_MASTER_KEY}
  database_url: "postgresql://..." # 可选，用于LiteLLM自身的用量追踪
```

```

model_list:
- model_name: gpt-4o
  litellm_params:
    model: openai/gpt-4o
    api_key: os.environ/OPENAI_API_KEY

- model_name: gpt-4o-mini
  litellm_params:
    model: openai/gpt-4o-mini
    api_key: os.environ/OPENAI_API_KEY

- model_name: deepseek-chat
  litellm_params:
    model: deepseek/deepseek-chat
    api_key: os.environ/DEEPSEEK_API_KEY

- model_name: claude-3.5-sonnet
  litellm_params:
    model: anthropic/claude-3-5-sonnet-20241022
    api_key: os.environ/ANTHROPIC_API_KEY

- model_name: gemini-2.5-pro
  litellm_params:
    model: gemini/gemini-2.5-pro-exp-03-25
    api_key: os.environ/GEMINI_API_KEY

litellm_settings:
  drop_params: true          # 忽略不支持的参数，防止报错
  set_verbose: false
  request_timeout: 120      # 超时2分钟

```

VPS安全配置：

1. LiteLLM 端口（4000）仅对 Worker 的出口IP开放
2. 防火墙：ufw allow from <Cloudflare_IP_Ranges> to any port 4000
3. 使用 Cloudflare Tunnel（cloudflared）做安全隧道，不暴露VPS IP

5.3 支付集成（PayJS）

MVP方案：PayJS 个人版，API直通车模式。

集成流程：

1. 注册 PayJS 商户号 → 获取 mchid + key
2. 前端生成充值订单 → 调 Worker API /api/topup/create
3. Worker 生成 PayJS 支付链接 → 返回给前端
4. 前端跳转到 PayJS 支付页（或展示二维码）
5. 用户扫码支付 → PayJS 异步回调 /api/topup/callback
6. Worker 验证签名 → D1 增加余额 + 记录充值

PayJS API 调用示例（Worker 端）：

```

async function createTopupOrder(userId: string, amount: number): Promise<string> {
  const params = {
    mchid: PAYJS_MCHID,
    total_fee: Math.round(amount * 100), // 分
    out_trade_no: `TOPUP-${Date.now()}-${userId.slice(0,8)}`,
    body: `AI家AI户 API充值 ¥${amount}`,
    notify_url: 'https://myaishome.com/api/topup/callback',
  };

  params.sign = generatePayJSSign(params, PAYJS_KEY);

  const response = await fetch('https://payjs.cn/api/native', {
    method: 'POST',
    body: new URLSearchParams(params),
  });

```

```
});

const data = await response.json();
return data.code_url; // 支付二维码链接
}
```

5.4 后端新建汇总

模块	技术栈	文件数	工作量	依赖
Worker项目框架	Hono + TypeScript	5	1天	wrangler环境
鉴权模块	bcryptjs + jose	3	1.5天	D1 users表
API路由(/v1)	Hono Router	2	1天	LiteLLM就绪
用户管理路由	Hono Router	3	1.5天	鉴权模块
计费引擎	TypeScript	2	1天	定价表确定
流式代理	TypeScript + ReadableStream	1	1.5天	LiteLLM就绪
支付集成	PayJS API	1	1天	PayJS商户号
LiteLLM部署	Docker + YAML	3	1天	VPS购买
D1数据库	SQL + wrangler CLI	3	0.5天	wrangler环境
合计		23	10天	

6. 数据库设计

6.1 表结构 (Cloudflare D1)

```
-- =====
-- 1. 用户表
-- =====
CREATE TABLE users (
  id          TEXT PRIMARY KEY,          -- UUID v4
  email       TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,          -- bcrypt(sha256(password))
  balance     REAL DEFAULT 0.0,         -- 余额 (人民币元)
  created_at  INTEGER NOT NULL,         -- Unix timestamp (秒)
  last_login_at INTEGER
);

-- =====
-- 2. API Keys 表
-- =====
CREATE TABLE api_keys (
  id          TEXT PRIMARY KEY,          -- UUID v4
  user_id     TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  key_hash    TEXT UNIQUE NOT NULL,     -- SHA-256(完整Key)
  prefix      TEXT NOT NULL,            -- "my-" + 前8位明文, UI展示
  name        TEXT DEFAULT '默认密钥',  -- 用户给key起的名字
  is_active   INTEGER DEFAULT 1,        -- 0=禁用, 1=启用
  created_at  INTEGER NOT NULL,
  last_used_at INTEGER
);

CREATE INDEX idx_api_keys_user ON api_keys(user_id);

-- =====
-- 3. 充值记录表
-- =====
CREATE TABLE topups (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id),
```

```

amount      REAL NOT NULL,          -- 充值金额（元）
bonus       REAL DEFAULT 0.0,       -- 赠送金额（元）
total       REAL NOT NULL,          -- amount + bonus
method      TEXT NOT NULL,          -- 'payjs' | 'manual'
trade_no    TEXT,                   -- PayJS 订单号
status      TEXT NOT NULL DEFAULT 'pending', -- pending | completed | failed
created_at  INTEGER NOT NULL
);

CREATE INDEX idx_topups_user ON topups(user_id);
CREATE INDEX idx_topups_trade ON topups(trade_no);

-- =====
-- 4. 用量日志表
-- =====
CREATE TABLE usage_logs (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL REFERENCES users(id),
  api_key_prefix TEXT NOT NULL,      -- "my-xxxx" 前缀
  model       TEXT NOT NULL,         -- 'gpt-4o'
  provider    TEXT NOT NULL,         -- 'openai'
  prompt_tokens INTEGER NOT NULL,
  completion_tokens INTEGER NOT NULL,
  cost        REAL NOT NULL,         -- 扣除金额（元）
  request_id  TEXT,                  -- 上游API返回的request_id（用于排错）
  created_at  INTEGER NOT NULL
);

CREATE INDEX idx_usage_user_date ON usage_logs(user_id, created_at DESC);
CREATE INDEX idx_usage_model ON usage_logs(model, created_at);

```

```

-- =====
-- 5. Guest试用记录表（临时，KV也可替代）
-- =====
CREATE TABLE guest_trials (
  guest_id     TEXT PRIMARY KEY,      -- 前端生成的UUID
  trial_count  INTEGER DEFAULT 0,
  last_trial_at INTEGER,
  created_at  INTEGER NOT NULL
);

```

6.2 数据流

```

用户注册 → INSERT users
      ↓
用户充值 → INSERT topups (pending) → 支付成功 → UPDATE topups + UPDATE users.balance
      ↓
用户创建Key → INSERT api_keys (生成时展示明文一次)
      ↓
用户调API → SELECT users (校验余额) → 转发LiteLLM → UPDATE users.balance (扣费) + INSERT usage_logs
      ↓
用户查用量 → SELECT usage_logs WHERE user_id + GROUP BY model + SUM(cost)

```

6.3 API Key 生成规则

格式: my-{32位随机字符}
 字符集: a-z A-Z 0-9
 示例: my-aB3xK9mQwR7tY2vL5nP8jF4dH6sA1cU0

生成逻辑:

1. `crypto.randomBytes(24).toString('base64url')` → 32字符
2. 前缀 my-
3. 数据库存 SHA-256(完整Key)
4. 响应中返回完整Key一次
5. prefix列存 "my-" + 前8字符 → UI展示 "my-aB3x****"

7. API接口设计

7.1 OpenAI兼容接口

GET /api/v1/models

```
// 响应
{
  "object": "list",
  "data": [
    {
      "id": "gpt-4o",
      "object": "model",
      "created": 1715367049,
      "owned_by": "openai"
    },
    {
      "id": "deepseek-chat",
      "object": "model",
      "created": 1700000000,
      "owned_by": "deepseek"
    }
  ]
}
```

POST /api/v1/chat/completions

```
// 请求（与OpenAI完全一致）
{
  "model": "gpt-4o",
  "messages": [
    { "role": "system", "content": "你是一个有用的助手" },
    { "role": "user", "content": "你好" }
  ],
  "temperature": 0.7,
  "max_tokens": 2048,
  "stream": true
}
```

```
// 响应（非流式）
{
  "id": "chatcmpl-xxx",
  "object": "chat.completion",
  "created": 1715367049,
  "model": "gpt-4o",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "你好！有什么我可以帮助你的吗？"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 20,
    "completion_tokens": 12,
    "total_tokens": 32
  }
}
```

```
// 响应（流式SSE）
data: {"id":"chatcmpl-xxx","object":"chat.completion.chunk","choices":[{"delta":{"role":"assistant"},"index":0}]}
data: {"id":"chatcmpl-xxx","object":"chat.completion.chunk","choices":[{"delta":{"content":"你"},"index":0}]}
data: {"id":"chatcmpl-xxx","object":"chat.completion.chunk","choices":[{"delta":{"content":"好"},"index":0}]}
data: {"id":"chatcmpl-xxx","object":"chat.completion.chunk","choices":[{"delta":{"finish_reason":"stop"},"index":0}]}
data: [DONE]
```

7.2 认证接口

POST /api/auth/register

```
// 请求
{
  "email": "user@example.com",
  "password": "mypassword123"
}

// 响应（成功 → 自动登录）
{
  "token": "eyJhbGciOi...",
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "balance": 0
  }
}

// 响应（失败）
{
  "error": "邮箱已被注册"
}
```

POST /api/auth/login

```
// 请求
{
  "email": "user@example.com",
  "password": "mypassword123"
}

// 响应
{
  "token": "eyJhbGciOi...",
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "balance": 9.9
  }
}
```

7.3 用户管理接口（需JWT）

GET /api/user/balance

```
// 响应
{
  "balance": 23.50,
  "total_used": 12.30,           // 累计消费
  "month_used": 5.20,           // 本月消费
  "month_tokens": {
    "gpt-4o": {"prompt": 200000, "completion": 80000},
    "deepseek-chat": {"prompt": 400000, "completion": 60000}
  }
}
```

GET /api/user/keys

```
// 响应
{
  "keys": [
    {
      "id": "key-uuid-1",
      "prefix": "my-aB3x****",
      "name": "默认密钥",
      "is_active": true,
      "created_at": 1715367049,
      "last_used_at": 1715367100
    }
  ]
}
```

POST /api/user/keys

```
// 请求
{
  "name": "项目A专用" // 可选
}

// 响应（仅此一次返回完整Key！）
{
  "id": "key-uuid-2",
  "full_key": "my-xY9mK3wQrT7vL2nP5jF8dH6sA1cU0bR",
  "prefix": "my-xY9m****",
  "name": "项目A专用",
  "created_at": 1715367200
}
```

DELETE /api/user/keys/:id

```
// 响应
{ "success": true }
```

GET /api/user/usage?limit=20&offset=0

```
// 响应
{
  "logs": [
    {
      "id": "log-uuid",
      "model": "gpt-4o",
      "prompt_tokens": 3200,
      "completion_tokens": 800,
      "cost": 0.032,
      "created_at": 1715367049
    }
  ],
  "total": 156
}
```

7.4 支付接口

POST /api/topup/create

```
// 请求（需JWT）
{
  "amount": 9.9
}

// 响应
{
  "order_id": "TOPUP-1715367049-aB3x",
  "amount": 9.9,
  "bonus": 0,
  "total": 9.9,
  "pay_url": "https://payjs.cn/api/cashier?..." // 移动端跳转
}
```

POST /api/topup/callback

```
// PayJS异步回调（无需JWT，靠签名验证）
// PayJS POST form data:
// mchid, out_trade_no, total_fee, transaction_id, sign, ...
// Worker验证 sign → 更新balance → 返回 success
```

7.5 错误响应格式

所有错误统一格式：

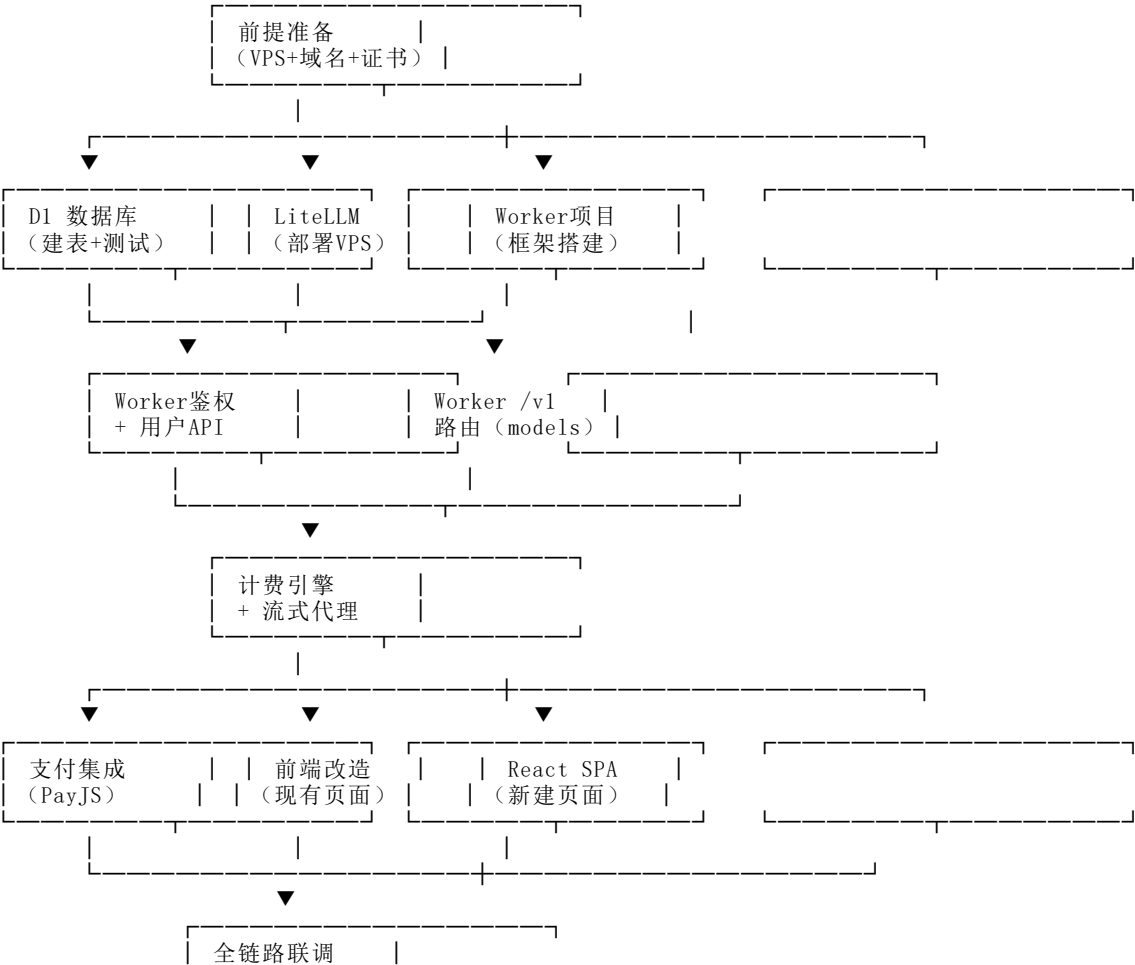
```
{
  "error": {
    "type": "insufficient_balance",
    "message": "余额不足，当前余额 ¥0.32，需充值后才能继续使用",
    "code": 402
  }
}
```

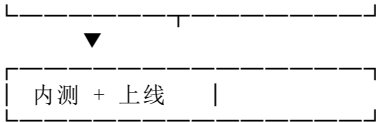
错误类型表：

Type	HTTP Code	说明
invalid_api_key	401	API Key无效或已禁用
insufficient_balance	402	余额不足
rate_limit_exceeded	429	频率限制
invalid_request	400	请求参数错误
model_not_found	404	模型未接入
internal_error	500	服务器内部错误
upstream_error	502	上游API返回错误

8. 开发步骤与里程碑

8.1 依赖关系图





8.2 详细开发排期（6周 × 5工作日 = 30工作日）

准备阶段（上线前1周，工作日-5~0）

天	任务	负责人	产出
-5	购买VPS（阿里云香港 2C4G）	个人	VPS IP + SSH密钥
-5	注册PayJS商户号	个人	mchid + key
-4	注册上游API（OpenAI/Anthropic/DeepSeek/Gemini）并充值	个人	5个API Key
-3	Cloudflare D1 创建 + wrangler 环境配置	开发	D1数据库ID
-3	域名DNS配置（api子域名 + Worker路由）	开发	DNS生效
-2	VPS基础环境：Docker + UFW + Cloudflare Tunnel	开发	VPS就绪
-1	Git仓库创建（ai-home-worker 新仓库或 monorepo）	开发	仓库就绪

第1周：基础设施 + 鉴权（工作日1-5）

天	任务	详细说明	产出
1	D1建表 + Worker项目框架	wrangler d1 execute 执行schema.sql，搭建Hono框架，配置wrangler.toml	5张表就绪，/api/health可访问
2	鉴权模块开发	实现bcrypt密码哈希、JWT签发/验证、API Key生成/校验	auth.ts + crypto.ts + jwt.ts
3	用户注册/登录API	POST /api/auth/register + /api/auth/login，单元测试	/api/auth/* 可调
4	用户管理API	/api/user/keys CRUD + /api/user/balance查询	用户API就绪
5	LiteLLM部署	VPS上 docker-compose up，配置5个模型，测试与上游API的连通性	LiteLLM可转发请求

第1周里程碑：✅ 用户可注册登录，LiteLLM可转发请求

第2周：核心API + 计费（工作日6-10）

天	任务	详细说明	产出
6-	Worker ↔ LiteLLM	实现proxy.ts，非流式请求全链路：Worker鉴权→转发→解析usage→扣费→记录日志	非流式全链路通
7	对接		
8	计费引擎	实现billing.ts：模型定价表 + calculateCost() + deductAndLog()	计费逻辑完成
9	流式代理	实现handleStreamingResponse：SSE透传 + 累计token + 结束扣费	流式全链路通
10	/api/v1/models	返回可用模型列表，含价格信息	/v1/models可用

第2周里程碑：✅ 用户可通过API Key调用模型，正常计费扣款

第3周：前端静态页改造（工作日11-15）

天	任务	详细说明	产出
11	index.html 改造	Hero区新增API CTA双按钮 + 样式	首页API入口就绪
12	models.html 改造 + 数据字段	卡片新增「试用API」按钮 + 「仅显示API」筛选 + models-data.js 新增has_api字段	模型库API入口就绪
13	44个详情页批量改造	修改generate_model_pages.py模板，重新生成全部详情页	详情页API入口就绪
14	api.html 页面开发	API Hub价目表 + curl/Python/Node.js代码示例 + FAQ	/api页面就绪
15	auth.html + compare.html改造	登录/注册页面 + 对比表API行	认证页+对比完成

第3周里程碑：✅ 现有站所有入口就绪，API Hub页面上线

第4周：React SPA + 支付（工作日16-20）

天	任务	详细说明	产出
16-17	Playground React SPA	项目搭建(Vite+React) + 6个组件开发 + SSE流式渲染 + Markdown 渲染	Playground可用
18-19	Dashboard React SPA	项目搭建 + 7个组件开发 + 与Worker API联调	Dashboard可用
20	支付集成	Worker端PayJS对接 + Dashboard充值入口 + 回调处理	支付闭环打通

第4周里程碑：✅ Playground和Dashboard可用，支付可充值

第5周：联调 + 测试（工作日21-25）

天	任务	详细说明	产出
21	全链路联调（前端→Worker→LiteLLM→上游）	注册→充值→创建Key→Playground测试→Dashboard查用量→余额扣减	全流程可用
22	功能测试 + 边界测试	余额不足、Key禁用、频率限制、超大prompt、特殊字符	测试报告
23	安全审计	Key泄露防护、SQL注入、XSS、JWT过期、密码强度	安全清单
24	性能测试	并发10/50/100请求，流式延迟，Worker CPU时间	性能报告
25	Bug修复 + 文档补全	修复测试发现的问题，API文档最终版	Bug清零

第5周里程碑：✅ 系统稳定，通过安全和性能测试

第6周：上线 + 推广（工作日26-30）

天	任务	详细说明	产出
26	部署上线	Worker生产环境配置、环境变量注入、Pages部署React SPA	正式上线
27	监控告警配置	Cloudflare Analytics + Worker错误日志 + 余额异常告警	监控就绪
28	推广准备	知乎/小红书文案 + 开发者社区推广（V2EX/掘金）	推广物料

天	任务	详细说明	产出
29	小范围内测	邀请10人内测，收集反馈	反馈列表
30	正式开放 + 推广发布	发布推广文案，监控首日数据	上线完成
第6周里程碑： ☒ 正式开放注册，监控就绪，推广启动			

8.3 里程碑总结

里程碑	周	验证方式
M1: 基础设施就绪	W0	D1可查，LiteLLM curl可调
M2: 鉴权+核心API	W1	注册/登录/Key生成可用
M3: 计费闭环	W2	API调用后余额正确扣减
M4: 前端入口	W3	所有页面有API入口
M5: 完整体验	W4	Playground可用，支付可充值
M6: 系统稳定	W5	测试通过，Bug清零
M7: 正式上线	W6	开放注册，监控运行

9. 测试计划

9.1 单元测试

模块	测试用例	覆盖目标
密码哈希	bcrypt正确性、不同密码不同哈希	100%
JWT	签发、验证、过期、篡改	100%
API Key生成	格式正确、无碰撞、哈希存储	100%
计费引擎	各种模型、0 token、负余额、精度	100%
PayJS签名	签名生成、回调验证	100%

9.2 集成测试

场景	步骤	预期结果
注册→登录→创建Key	1) POST /register 2) POST /login 3) POST /keys	Key返回完整一次
充值→调用→扣费	1) 充值¥9.9 2) 调chat 3) 查balance	balance减少=usage cost
余额不足	1) 余额清零 2) 调chat	402错误
Key禁用	1) 禁用Key 2) 用该Key调chat	401错误
流式响应	1) stream=true 2) 验证SSE格式 3) 查扣费	流式正常，扣费正确

9.3 安全测试

测试项	方法	通过标准
SQL注入	邮箱输入' OR 1=1 --	不返回非授权数据
XSS	prompt注入 <script>alert(1)</script>	不会执行

测试项	方法	通过标准
密码暴力破解	连续登录失败100次	IP被限流
JWT伪造	修改JWT payload后请求	401错误
Key遍历	尝试猜解API Key	限流 + 哈希不可逆

9.4 性能测试基准

指标	目标	测试方法
非流式延迟增量	< 100ms	Worker + LiteLLM vs 直连上游
流式首字节时间	< 500ms	SSE第一个data到达时间
并发10请求	100%成功, P95 < 2s wrk / k6	
Worker CPU时间	< 30ms/请求	Cloudflare Analytics
D1查询时间	< 5ms	wrangler tail

10. 部署与运维方案

10.1 部署架构

生产环境:

- Cloudflare Pages (myaishome.com)
 - 静态页面 (index.html, models.html, api.html, auth.html ...)
 - React SPA (/playground/*, /dashboard/*)
 - Worker路由 (/api/* → Worker)
- Cloudflare Worker (api.myaishome.com 或同域名 /api/*)
 - D1 数据库绑定
 - 环境变量: LITELLM_URL, PAYJS_KEY, JWT_SECRET ...
- VPS (阿里云香港)
 - Docker: LiteLLM (内网:4000)
 - Docker: cloudflared (安全隧道, 不暴露VPS IP)
 - UFW防火墙: 仅允许Cloudflare IP入站

10.2 CI/CD

GitHub Actions 工作流:

```
# .github/workflows/deploy.yml

# 静态站部署 (现有流程不变)
deploy-pages:
  on: push to main
  steps:
    - Cloudflare Pages deploy

# Worker部署
deploy-worker:
  on: push to main, paths: 'ai-home-worker/**'
  steps:
    - cd ai-home-worker
    - npm install
    - wrangler deploy

# React SPA构建 + 部署
deploy-spa:
  on: push to main, paths: 'spa/**'
  steps:
    - cd spa/playground && npm run build
```

10.3 监控指标

指标	工具	告警阈值
Worker错误率	Cloudflare Analytics	> 1%
LiteLLM可用性	Uptime监控 (cron job)	连续3次失败
D1查询延迟	wrangler tail	P95 > 50ms
余额异常	自定义告警	单用户余额突降 > ¥100/小时
上游API故障	Worker错误日志	502错误量 > 10次/分钟

10.4 备份策略

数据	频率	方式
D1数据库	每日	wrangler d1 backup → R2存储
Worker代码	每次push	GitHub版本控制
LiteLLM配置	每次变更	Git + VPS备份

11. 风险与应对措施

风险	概 率	影 响	应对措施
上游API涨价	中	高	定价模型中预留20%调价空间；涨价时提前通知用户，同步调整售价
上游API故障	中	中	Worker层做健康检查 + 自动降级（如GPT-4o挂了自动切GPT-4o-mini告知用户）
VPS被墙/故障	低	高	备选VPS（腾讯云新加坡）+ Cloudflare Tunnel自动切换
有人滥用API	中	中	频率限制 + 余额门槛 + 异常检测自动封Key
支付风控	中	中	PayJS有基础风控；异常退款人工处理；保留30%现金准备金
数据安全事件	低	高	不存prompt内容、Key只存哈希、定期安全审计
Cloudflare被墙（国内）	低	高	备用国内CDN方案（如又拍云）；仅影响国内用户
法律合规	中	中	用户协议明确禁止生成违法内容；接入敏感词过滤

12. 附录

12.1 项目仓库规划

仓库	内容	说明
gyuanhao/ai-home（现有）	静态站 + React SPA 构建产物	保持不变
gyuanhao/ai-home-worker（新建）	Cloudflare Worker 代码	独立仓库或 monorepo 子目录

12.2 环境变量清单

Worker 环境变量：

```
# 数据库
D1_DATABASE_ID=xxx

# LiteLLM连接
LITELLM_URL=https://litellm.internal.myaishome.com
LITELLM_MASTER_KEY=sk-litellm-xxx

# JWT
JWT_SECRET=随机生成的64位字符串

# 支付
PAYJS_MCHID=商户号
PAYJS_KEY=支付密钥

# 频率限制
RATE_LIMIT_FREE=10      # 免费用户次/分钟
RATE_LIMIT_PAID=60      # 付费用户次/分钟
```

LiteLLM 环境变量 (.env，不提交Git)：

```
OPENAI_API_KEY=sk-proj-xxx
ANTHROPIC_API_KEY=sk-ant-xxx
DEEPSEEK_API_KEY=sk-xxx
GEMINI_API_KEY=AIzaSy...
LITELLM_MASTER_KEY=sk-litellm-xxx
```

12.3 项目总工作量

阶段	工作日	主要产出
准备阶段	5天	VPS、API Key、D1、仓库
W1 基础设施	5天	鉴权 + LiteLLM部署
W2 核心API	5天	计费 + 流式代理
W3 前端改造	5天	静态页面入口 + API Hub
W4 SPA + 支付	5天	Playground + Dashboard + 支付
W5 测试	5天	联调 + 安全 + 性能
W6 上线	5天	部署 + 监控 + 推广
合计	30工作日	

12.4 关键依赖的npm包

Worker端：

```
{
  "dependencies": {
    "hono": "^4.x",
    "jose": "^5.x",
    "bcryptjs": "^2.x"
  },
  "devDependencies": {
    "@cloudflare/workers-types": "^4.x",
    "wrangler": "^3.x",
    "typescript": "^5.x"
  }
}
```

React SPA端：

```
{
  "dependencies": {
```

```
    "react": "^18.x",
    "react-dom": "^18.x",
    "react-router-dom": "^6.x",
    "marked": "^12.x",
    "highlight.js": "^11.x"
  },
  "devDependencies": {
    "vite": "^5.x",
    "@vitejs/plugin-react": "^4.x",
    "typescript": "^5.x"
  }
}
```

12.5 参考资源

- Cloudflare Workers 文档: <https://developers.cloudflare.com/workers/>
- Cloudflare D1 文档: <https://developers.cloudflare.com/d1/>
- LiteLLM GitHub: <https://github.com/BerriAI/litellm>
- Hono 框架: <https://hono.dev/>
- PayJS 文档: <https://payjs.cn/help>
- OpenAI API 文档: <https://platform.openai.com/docs/api-reference>

文档版本: v1.0 | **最后更新:** 2026-07-02

下一步: 确认准备阶段事项 → 开始 W0 准备工作 (购买VPS、注册API Key)